

Assignment 12: Unsupervised Machine Learning with Python

Name Ayleen Santander

1. Data Selection

```
#Libraries required
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly as py
import plotly.graph_objs as go
import warnings
import hdbscan
from scipy.cluster.hierarchy import fcluster
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import HDBSCAN
warnings.filterwarnings("ignore")
```

1. Perform exploratory data analysis on the Wine dataset.

```
from sklearn.datasets import load_wine
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target, name='class')
X.head()
```

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	col
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	

Next steps:

[Generate code with X](#)
[New interactive sheet](#)

(y.head())

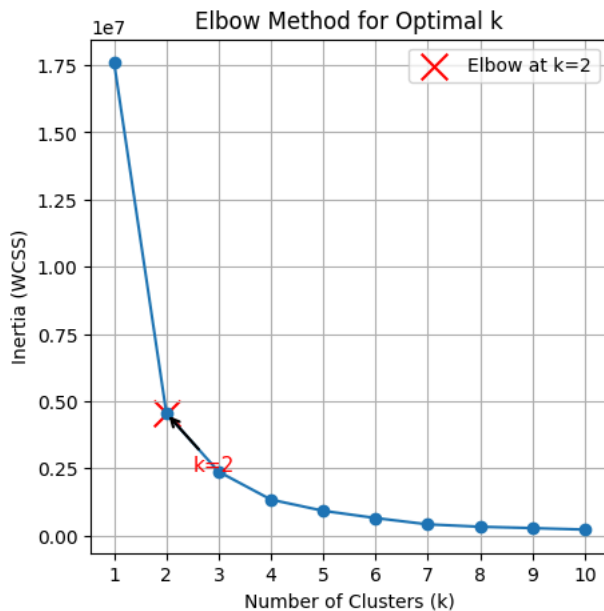
	class
0	0
1	0
2	0
3	0
4	0

dtype: int64

2. Implement at least two clustering algorithms (e.g., DBSCAN and Hierarchical Clustering) to find patterns in the data

✓ Clustering algorithm= 1. Kmeans

```
# Applying inertia to find the optimal k, Elbow Method
inertia = []
k_range = range(1,11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42,n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
plt.figure(figsize=(5, 5))
plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method for Optimal k')
plt.xticks(k_range)
plt.grid(True)
# Mark the elbow point (k=2)
optimal_k = 2
plt.scatter(optimal_k, inertia[optimal_k-1], color='red', marker='x', s=200, label=f'Elbow at k={optimal_k}')
plt.annotate(f'k={optimal_k}', xy=(optimal_k, inertia[optimal_k-1]),xytext=(optimal_k + 0.5, inertia[optimal_k] + 0.3),
            fontsize=11, color='red', arrowprops=dict(facecolor='red', arrowstyle='->', lw=1.5))
plt.legend()
plt.show()
```



```
#Applying KMeans for k=2
kmeans = KMeans(n_clusters=2, init='random', n_init='auto', max_iter=300)
y_kmeans = kmeans.fit_predict(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

```
#define the number of data points in each cluster
X['Cluster'] = pd.DataFrame(y_kmeans, columns=['labels'])
print("Number of data points in each cluster=\n", X['Cluster'].value_counts())
X.head()
```

Number of data points in each cluster=

Cluster

0 123

1 55

Name: count, dtype: int64

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	col
0	14.23	1.71	2.43		15.6	127.0	2.80	3.06	0.28	2.29
1	13.20	1.78	2.14		11.2	100.0	2.65	2.76	0.26	1.28
2	13.16	2.36	2.67		18.6	101.0	2.80	3.24	0.30	2.81
3	14.37	1.95	2.50		16.8	113.0	3.85	3.49	0.24	2.18
4	13.24	2.59	2.87		21.0	118.0	2.80	2.69	0.39	1.82

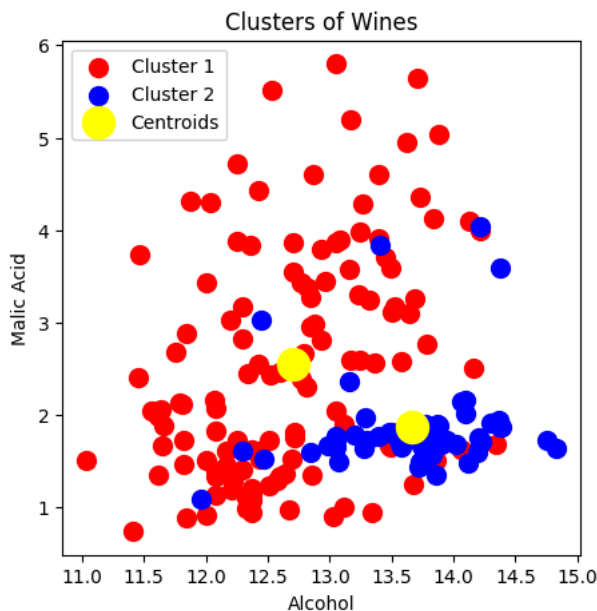
Next steps:

[Generate code with X](#)

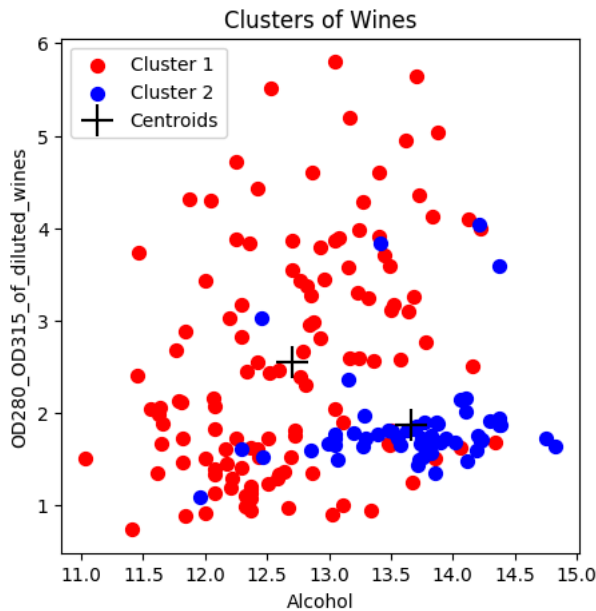
[New interactive sheet](#)

```
# Visualizing the clusters
plt.figure(figsize=(5, 5))
X_array = X.values
plt.scatter(X_array[y_kmeans == 0, 0], X_array[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X_array[y_kmeans == 1, 0], X_array[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')

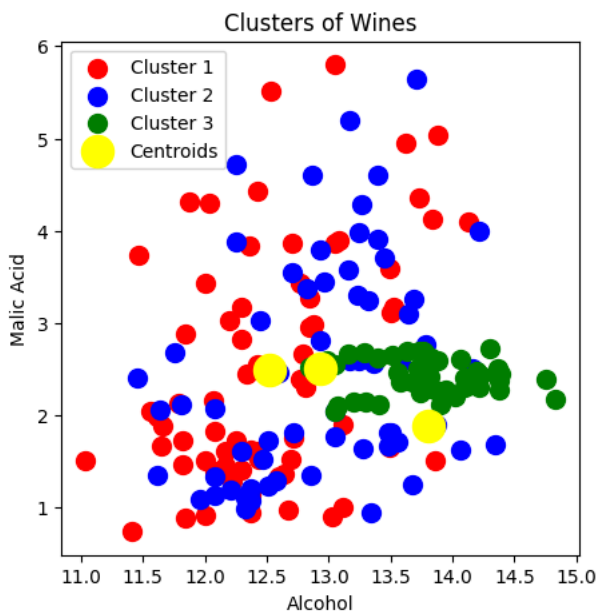
# Plotting the centroids of the clusters
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='yellow', label='Centroids')
plt.title('Clusters of Wines')
plt.xlabel('Alcohol')
plt.ylabel('Malic Acid')
plt.legend()
plt.show()
```



```
# Plot the centroid and cluster with different colors to visualize
# how k-means algorithm has grouped the data
plt.figure(figsize=(5, 5))
plt.scatter(X.iloc[y_kmeans == 0, 0], X.iloc[y_kmeans == 0, 1], s=50, c='red', label='Cluster 1')
plt.scatter(X.iloc[y_kmeans == 1, 0], X.iloc[y_kmeans == 1, 1], s=50, c='blue', label='Cluster 2')
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, marker='+', c='black', label='Centroids')
plt.title('Clusters of Wines')
plt.xlabel('Alcohol')
plt.ylabel('OD280_OD315_of_diluted_wines')
plt.legend()
plt.show()
```



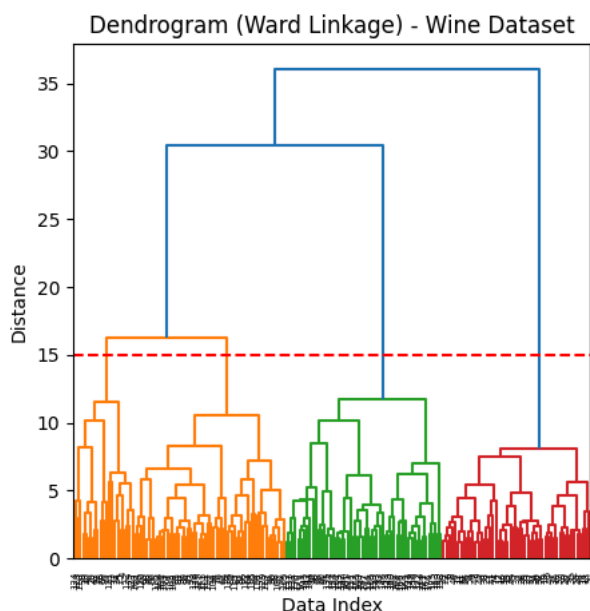
```
#Applying KMeans for k=3 to matcht with the 3 cultivars in the data
kmeans = KMeans(n_clusters=3, init='random', n_init='auto', max_iter=300)
y_kmeans = kmeans.fit_predict(X)
labels =kmeans.labels_
centroids =kmeans.cluster_centers_
# Visualizing the clusters
plt.figure(figsize=(5, 5))
X_array = X.values
plt.scatter(X_array[y_kmeans == 0, 0], X_array[y_kmeans == 0, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X_array[y_kmeans == 1, 0], X_array[y_kmeans == 1, 1], s=100, c='blue', label='Cluster 2')
plt.scatter(X_array[y_kmeans == 2, 0], X_array[y_kmeans == 2, 1], s=100, c='green', label='Cluster 3')
# Plotting the centroids of the clusters
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s=300, c='yellow', label='Centroids')
plt.title('Clusters of Wines')
plt.xlabel('Alcohol')
plt.ylabel('Malic Acid')
plt.legend()
plt.show()
```



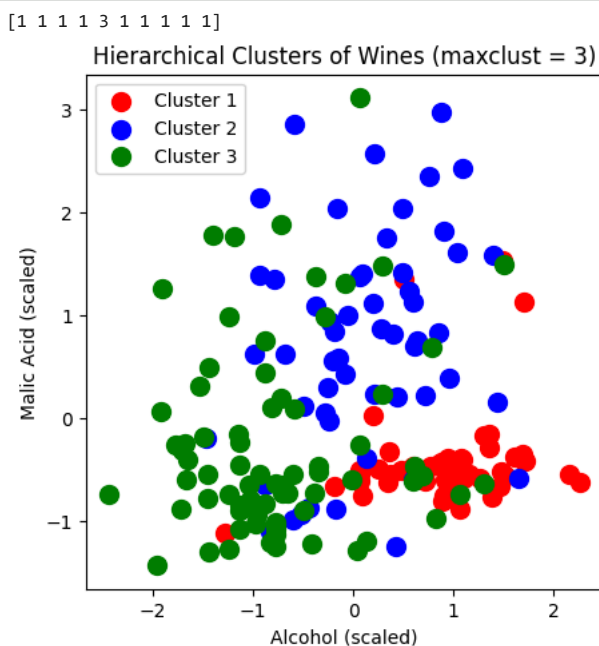
✧ Clustering Algorithm= 2. Hierarchical Clustering + Dendrogram

```
#Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # X is your DataFrame from load_wine()
```

```
# Hierarchical Clustering + Dendrogram
linked = linkage(X_scaled, method='ward')
plt.figure(figsize=(5, 5))
dendrogram(linked, orientation='top', distance_sort='descending', show_leaf_counts=True)
plt.title('Dendrogram (Ward Linkage) - Wine Dataset')
plt.xlabel('Data Index')
plt.ylabel('Distance')
plt.axhline(y=15, color='red', linestyle='--', label='Cut line') # (3clusters)
plt.show()
```

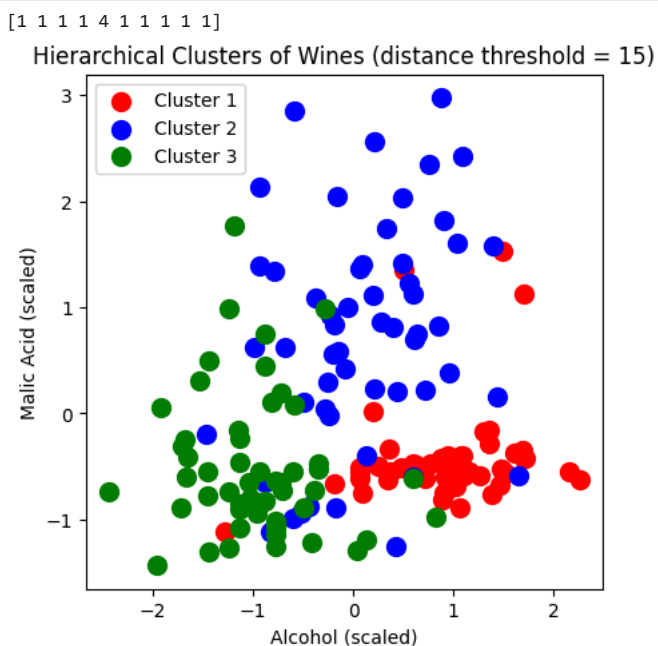


```
# Option A: cut by number of clusters
n_clusters = 3 # based on your dendrogram inspection
hier_labels = fcluster(linked, t=n_clusters, criterion='maxclust')
print(hier_labels[:10]) # cluster for first 10 wines
plt.figure(figsize=(5, 5))
plt.scatter(X_scaled[hier_labels == 1, 0], X_scaled[hier_labels == 1, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X_scaled[hier_labels == 2, 0], X_scaled[hier_labels == 2, 1], s=100, c='blue', label='Cluster 2')
plt.scatter(X_scaled[hier_labels == 3, 0], X_scaled[hier_labels == 3, 1], s=100, c='green', label='Cluster 3')
plt.title('Hierarchical Clusters of Wines (maxclust = 3)')
plt.xlabel('Alcohol (scaled)')
plt.ylabel('Malic Acid (scaled)')
plt.legend()
plt.show()
```



```
# Option B: cut by distance threshold
hier_labels = fcluster(linked, t=15, criterion='distance')
print(hier_labels[:10]) # cluster

plt.figure(figsize=(5, 5))
plt.scatter(X_scaled[hier_labels == 1, 0], X_scaled[hier_labels == 1, 1], s=100, c='red', label='Cluster 1')
plt.scatter(X_scaled[hier_labels == 2, 0], X_scaled[hier_labels == 2, 1], s=100, c='blue', label='Cluster 2')
plt.scatter(X_scaled[hier_labels == 3, 0], X_scaled[hier_labels == 3, 1], s=100, c='green', label='Cluster 3')
plt.title('Hierarchical Clusters of Wines (distance threshold = 15)')
plt.xlabel('Alcohol (scaled)')
plt.ylabel('Malic Acid (scaled)')
plt.legend()
plt.show()
```



```
#Extract 3 cluster labels to match
hier_labels = fcluster(linked, t=3, criterion='maxclust')
hier_labels
```

```
array([ 1,  1,  1,  1,  3,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  3,  3,  3,  
       1,  1,  3,  3,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  3,  3,  1,  1,  3,  
       1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  3,  2,  2,  3,  3,  3,  3,  
       3,  3,  2,  3,  2,  3,  3,  3,  1,  3,  3,  3,  3,  3,  3,  2,  3,  3,  3,  3,  3,  
       3,  3,  3,  3,  3,  3,  3,  2,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3])
```

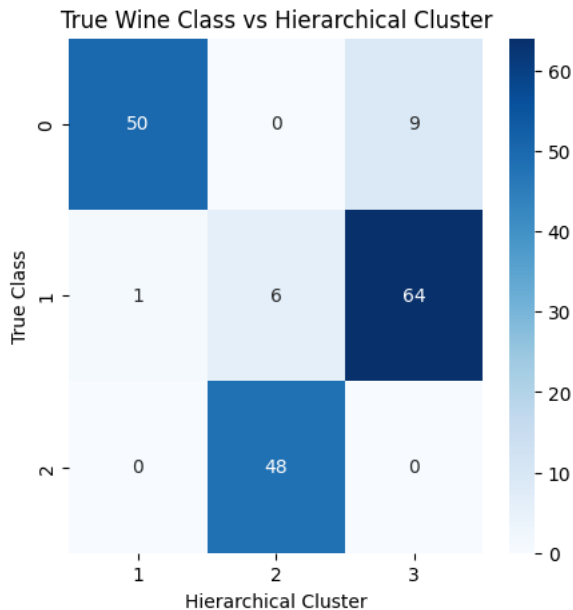
```
3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2], dtype=int32)
```

```
#check 3 hierarchical cluster align with the true wine class
comparison = pd.crosstab(y, hier_labels, rownames=['True Class'], colnames=['Hierarchical Cluster'])
print(comparison)
```

```
Hierarchical Cluster  1  2  3
True Class
0                   50  0  9
1                    1  6 64
2                    0 48  0
```

The output means: True Class =0 almost entirely landed in Cluster 1 (50 out of 59 total) with only 9 samples leaking into Cluster 3 True Class =1 almost entirely landed in Cluster 3 (64 out of total 71 total), with a small amount of overlap(6 in Cluster 2, 1 in Cluster 1) True Class =2, landed 100% in Cluster 2(48 out of 48 (a perfect match, no leakage at all))

```
#Visualize the hierarchical cluster align with the true wine class
plt.figure(figsize=(5, 5))
sns.heatmap(comparison, annot=True, fmt='d', cmap='Blues', cbar=True)
plt.title('True Wine Class vs Hierarchical Cluster')
plt.xlabel('Hierarchical Cluster')
plt.ylabel('True Class')
plt.show()
```



```
#Single number to Summarize this instead of eyeballing the table, use Adjusted Rand Index(ARI), which measures agreement between
ari = adjusted_rand_score(y, hier_labels)
print(f"Adjusted Rand Index: {ari:.3f}")
```

```
Adjusted Rand Index: 0.742
```

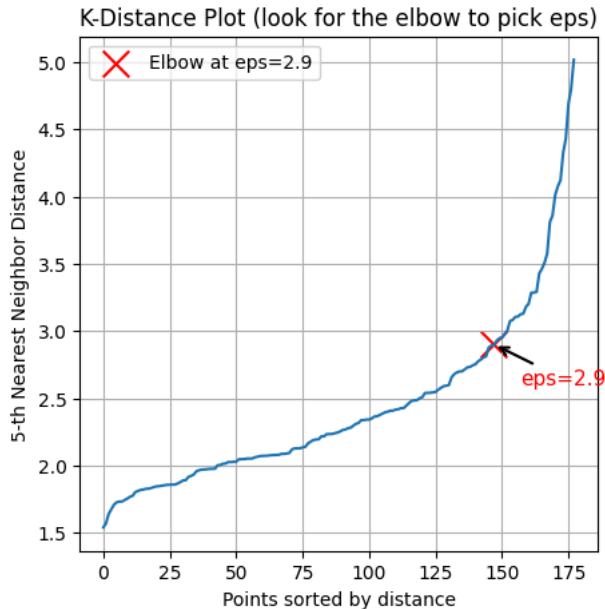
ARI tell us, ARI close to 1.0 confirms strong agreement(perfect match= 1.0, random labeling ~0.0)

Interpret the scale = 0.742 strong, well above chance solidly in the 'good clustering' range. My Hierarchical clustering agrees strongly with the true wine classes, but isn't a perfect match

✓ Clustering algorithm= 3. DBSCAN clustering

```
#Find the good eps /eps (ε): the maximum distance between two points for them to be considered neighbors/#classify the points =
min_samples = 5
neighbors = NearestNeighbors(n_neighbors=min_samples).fit(X_scaled)
distances, indices = neighbors.kneighbors(X_scaled)
```

```
#Sort distances to the k-th nearest neighbor, the plot
distances = np.sort(distances[:, min_samples - 1], axis=0)
plt.figure(figsize=(5, 5))
plt.plot(distances)
plt.xlabel('Points sorted by distance')
plt.ylabel(f'{min_samples}-th Nearest Neighbor Distance')
plt.title('K-Distance Plot (look for the elbow to pick eps)')
plt.grid(True)
# Pick a single eps value based on the elbow (e.g. 2.9, midpoint of your 2.8-3.0 estimate)
optimal_eps = 2.9
# Find the x-position (index) where the sorted distance is closest to optimal_eps
elbow_index = np.argmin(np.abs(distances - optimal_eps))
# Mark the elbow point
plt.scatter(elbow_index, optimal_eps, color='red', marker='x', s=200, label=f'Elbow at eps={optimal_eps}')
plt.annotate(f'eps={optimal_eps}', xy=(elbow_index, optimal_eps), xytext=(elbow_index + 10, optimal_eps - 0.3),
            fontsize=11, color='red', arrowprops=dict(facecolor='red', arrowstyle='->', lw=1.5))
plt.legend()
plt.show()
```



Analysis of the plot = The bend starts around x=150, where the y-axis (distance) is roughly 2.8- 3.0. that's the elbow, so reasonable starting point would be eps ~ 2.8 to 3.0.

```
#Load and convert Bunch = DF
wine= load_wine()
liquor= pd.DataFrame(wine.data, columns=wine.feature_names)
liquor['Class']= wine.target
#Run DBSCAN with the chosen eps
eps = 2.9 #Elbow result in DBSCAN algorithm
min_samples = 5
dbscan_eps = DBSCAN(eps=eps, min_samples=min_samples)
liquor['DBSCAN_Cluster'] = dbscan_eps.fit_predict(X_scaled)
print(liquor['DBSCAN_Cluster'].value_counts())
print(f'Noise vs DBSCAN Cluster')
print('-'*20)
print(pd.crosstab(liquor['Class'], liquor['DBSCAN_Cluster']))
```

```
DBSCAN_Cluster
0    164
-1    14
Name: count, dtype: int64
Noise vs DBSCAN Cluster
-----
DBSCAN_Cluster -1    0
Class
0                0    59
1                11    60
2                 3    45
```

```
# Plot 1: Scatter of clusters (noise in black)
```

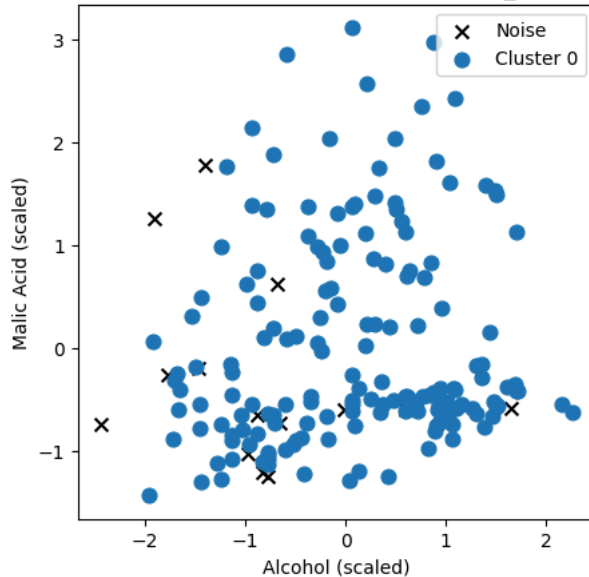


```

plt.figure(figsize=(5, 5))
unique_labels = sorted(liquor['DBSCAN_Cluster'].unique())
for label in unique_labels:
    mask = liquor['DBSCAN_Cluster'] == label
    if label == -1:
        # Noise points
        plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1],
                    s=50, c='black', marker='x', label='Noise')
    else:
        plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1],
                    s=60, label=f'Cluster {label}')
plt.title(f'DBSCAN Clustering of Wines (eps={eps}, min_samples={min_samples})')
plt.xlabel('Alcohol (scaled)')
plt.ylabel('Malic Acid (scaled)')
plt.legend()
plt.show()

```

DBSCAN Clustering of Wines (eps=2.9, min_samples=5)



```

#Plot 2: Heatmap of True Class vs DBSCAN Cluster
comparison = pd.crosstab(liquor['Class'], liquor['DBSCAN_Cluster'], rownames=['True Class'], colnames=['DBSCAN Cluster'])
plt.figure(figsize=(5, 5))
sns.heatmap(comparison, annot=True, fmt='d', cmap='Greens', cbar=True)
plt.title('True Wine Class vs DBSCAN Cluster')
plt.xlabel('DBSCAN Cluster')
plt.ylabel('True Class')
plt.show()

```



Interpretation = Value_counts (), (DBSCAN_Cluster) only found one real cluster(label = 0, containing 164 points) and labeled 14 points as noise (label=-1).

2nd result, Crosstab(df['Class'], df['DBSCAN_Cluster']): Class 0 (59 wines): non flagged as noise Class 1 (71 wines), 11 were flagged as noise (-1) Class 2 (48 wines), 3 flagged as noise This means DBSCAN lumped all three cultivars into one giant cluster, and only picked out 14 scattered points as outliers.

Compare this results with Kmeans (k=3) and Hierarchical clustering (Ward, 3 clusters), likely did a much better job separating the clases, because they don't require density gaps, they just look at distances to centroids or merge point progressively.

✓ Clustering Algorithm = 4. HDBSCAN

```
#Fit HDBSCAN
# min_cluster_size = smallest grouping you'll accept as a real cluster
# min_samples = controls how conservative the clustering is (higher = more points labeled noise)
hdb = HDBSCAN(min_cluster_size=5, min_samples=5)
liquor['HDBSCAN_Cluster'] = hdb.fit_predict(X_scaled)
```

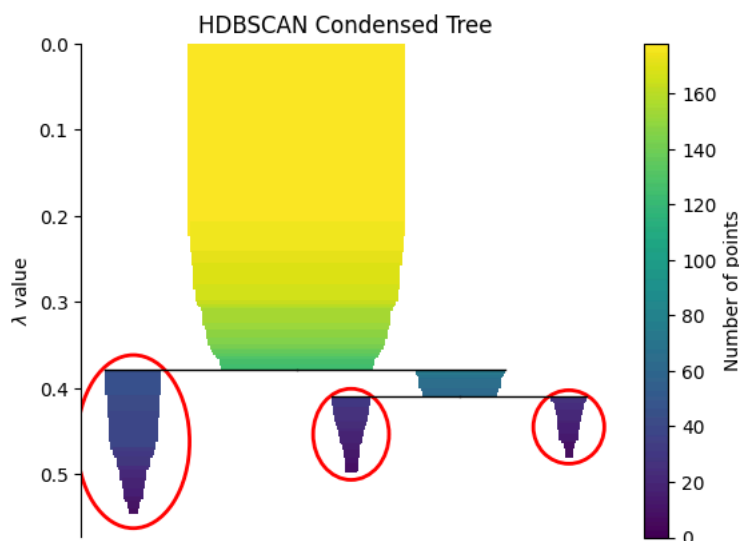
```
#Check cluster sizes and noise
print(liquor['HDBSCAN_Cluster'].value_counts())
```

```
HDBSCAN_Cluster
-1    63
 0    48
 2    34
 1    33
Name: count, dtype: int64
```

```
#Compare to true Class labels
print(pd.crosstab(liquor['Class'], liquor['HDBSCAN_Cluster']))
```

```
HDBSCAN_Cluster  -1    0    1    2
Class
0                11   48    0    0
1                37    0   33    1
2                15    0    0   33
```

```
hdb = hdbscan.HDBSCAN(min_cluster_size=5, min_samples=5)
liquor['HDBSCAN_Cluster'] = hdb.fit_predict(X_scaled)
#Inspect cluster stability/ condensed tree
hdb.condensed_tree_.plot(select_clusters=True)
plt.title('HDBSCAN Condensed Tree')
plt.show()
```

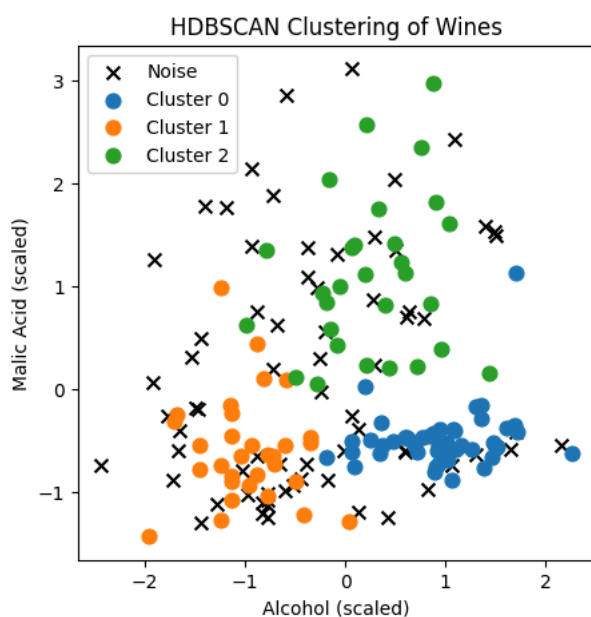


Interpretation = HDBSCAN found 3 stable clusters, the same number as the actual wine cultivars. The figure shows 3 clusters (red circle)

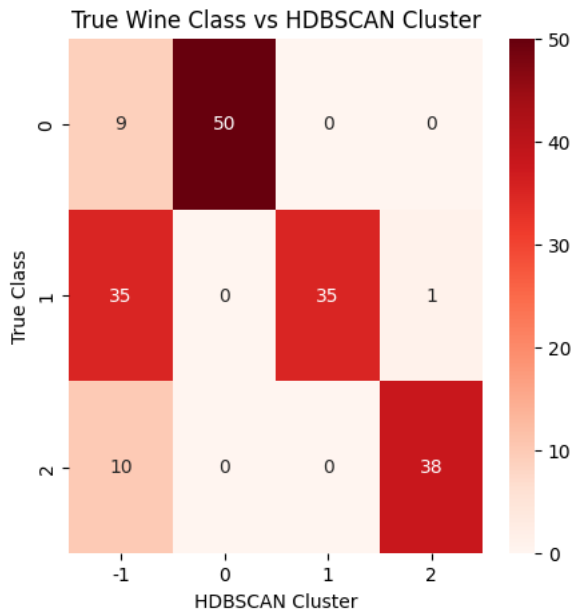
```
#Check the clusters actually line up with the real classes
print(liquor['HDBSCAN_Cluster'].value_counts())
print(pd.crosstab(liquor['Class'], liquor['HDBSCAN_Cluster']))
```

```
HDBSCAN_Cluster
-1    72
 0    46
 1    31
 2    29
Name: count, dtype: int64
HDBSCAN_Cluster -1    0    1    2
Class
0              13   46    0    0
1              39    0   31    1
2              20    0    0   28
```

```
#Scatter of clusters (noise in black)
plt.figure(figsize=(5, 5))
unique_labels = sorted(liquor['HDBSCAN_Cluster'].unique())
for label in unique_labels:
    mask = liquor['HDBSCAN_Cluster'] == label
    if label == -1:
        # Noise points
        plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1], s=50, c='black', marker='x', label='Noise')
    else:
        plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1], s=60, label=f'Cluster {label}')
plt.title('HDBSCAN Clustering of Wines')
plt.xlabel('Alcohol (scaled)')
plt.ylabel('Malic Acid (scaled)')
plt.legend()
plt.show()
```



```
#Plot 2: Heatmap of True Class vs HDBSCAN Cluster
comparison = pd.crosstab(liquor['Class'], liquor['HDBSCAN_Cluster'], rownames=['True Class'], colnames=['HDBSCAN Cluster'])
plt.figure(figsize=(5, 5))
sns.heatmap(comparison, annot=True, fmt='d', cmap='Reds', cbar=True)
plt.title('True Wine Class vs HDBSCAN Cluster')
plt.xlabel('HDBSCAN Cluster')
plt.ylabel('True Class')
plt.show()
```



Interpretation = -Cluster sizes=72 out of 178 points (40%) labeled as noise, a big chunk of the dataset didn't get assigned to any cluster at all. -Crosstab = shows 3 clusters and noise -Class 0 (46 points): 100% from Class 0 = Clean, pure cluster -Class 1 (32 points): 31 from Class 1, 0 from others, clean, pure cluster class 2 (29 points): 28 from Class 2, 1 from Class 1, nearly pure

Noise interpretation= -Class 0 (59 total), 46 correctly clustered, 13 called noise (22% of class 0 discarded) -Class 1 (71 total), 31 correctly clustered, 39 called noise (55% of class 1 discarded.) -Class 2 (48 total), 28 correctly clustered, 20 called noise (42% discarded) HDBSCAN is being very conservative, selected the point to a cluster is very confident and the rest throws into noise.

```
#HDBSCAN recall for precision
for ms in [3, 5, 8, 10]:
    hdb_test = hdbscan.HDBSCAN(min_cluster_size=5, min_samples=ms)
    labels = hdb_test.fit_predict(X_scaled)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)
    print(f"min_samples={ms}: {n_clusters} clusters, {n_noise} noise points")
```

```
min_samples=3: 3 clusters, 54 noise points
min_samples=5: 3 clusters, 72 noise points
min_samples=8: 3 clusters, 79 noise points
min_samples=10: 3 clusters, 85 noise points
```

Interpretation = the number of cluster keep at the same = 3, HDBSCAN is confidently detecting the true 3 cultivar structure.

- min_samples = give the best trade-off in this sweep; same 3 clusters , but the least noise (53 vs 72 I got before)

```
#Check it lowering min_sample hurt purity
hdb_final = hdbscan.HDBSCAN(min_cluster_size=5, min_samples=3)
liquor['HDBSCAN_Cluster'] = hdb_final.fit_predict(X_scaled)
print(liquor['HDBSCAN_Cluster'].value_counts())
print(pd.crosstab(liquor['Class'], liquor['HDBSCAN_Cluster']))
```

```
HDBSCAN_Cluster
-1    54
 0    50
 2    39
 1    35
Name: count, dtype: int64
HDBSCAN_Cluster -1    0    1    2
Class
0              9   50    0    0
1             35    0   35    1
2             10    0    0   38
```

Interpretation = Lowering min_samples from 5 to 3 recovered 18 more points from noise while preserving cluster purity.

Discussion

Application four unsupervised clustering techniques; K-Means, Hierarchical (Ward Linkage), DBSCAN, and HDBSCAN, to the UCI Wine Dataset (178 samples, 13 chemical attributes, 3 known cultivars), to evaluate how well each method could recover the true grouping structure without ever seeing the class labels.

- K-Means and Hierarchical Clustering both performed strongly. Using the Elbow Method, $k=2$ shows in the plot elbow bend in the $k=2$ that is the optimal point instead the dataset shows 3 cultivars. When I plot with $k=3$ even the elbow shows a different $k=2$, the plot gives me more sense maybe the split in 3 instead 2 is the best result and the distance is most consistently.
- DBSCAN = failed to separate the data meaningfully. With a single global 'eps' value, it treated nearly the entire dataset as one dense cluster and flagged only 14 points as noise, this is the limitation of this algorithm. In this case assumes clusters are separated by regions of low density, but the wine cultivars overlap enough chemically that no such gap exists at any single density threshold.